ЛАБОРАТОРНАЯ РАБОТА № 3

Основы обработки цифровых изображений

Отчет по варианту 8

Высокочастотная фильтрация и пороговая обработка

Стек: React + TypeScript

Документ описывает подготовленную реализацию и порядок проверки. Скриншоты, измерения времени и результаты фактических запусков не включены.

1. Цель работы

Закрепить знания об обработке цифровых изображений и разработать веб-приложение, которое позволяет загружать изображение, применять алгоритмы обработки и сравнивать исходное изображение с результатом.

2. Задание

По таблице вариантов необходимо реализовать высокочастотные фильтры для повышения резкости, два метода глобальной пороговой обработки и адаптивную пороговую обработку.

Выбранные методы: повышение резкости на основе Лапласиана с четырьмя и восемью соседями; глобальный порог Оцу; итеративный глобальный порог; адаптивный порог по локальному среднему.

Приложение должно иметь графический интерфейс и тестовую базу изображений. Для сдачи требуются исходный код, сопроводительная документация и опубликованное веб-приложение.

3. Средства и структура приложения

React используется для интерфейса, TypeScript — для описания данных и реализации алгоритмов. Vite запускает проект и формирует production-сборку. Canvas применяется для чтения RGBA-данных, отображения и сохранения PNG. Готовые функции фильтрации изображений не используются.

Обработка выполняется в одном Web Worker отдельно от интерфейса. Это уменьшает блокировку страницы, но не означает распараллеливание одного алгоритма на несколько вычислительных потоков.

• App.tsx — загрузка изображения, выбор метода и параметров, вывод результата.

• pixels.ts — пиксельный буфер, яркость, доступ к соседям и обработка краев.

• algorithms.ts — математические функции обработки без зависимости от React.

• worker.ts — вызов алгоритмов в фоновом потоке.

4. Описание методов

4.1. Повышение резкости

Для каждого канала RGB вычисляется разность между значением центрального пикселя и его соседями. Полученная высокочастотная составляющая прибавляется к исходному значению:

g = f + a · (n · f − сумма соседей)

Здесь f — исходное значение, g — результат, a — сила эффекта, n — число соседей. Для четырех соседей используются пиксели сверху, снизу, слева и справа. Для восьми соседей дополнительно используются диагональные пиксели.

При a = 1 ядро с четырьмя соседями имеет коэффициент 5 в центре, −1 сверху, снизу, слева и справа, 0 на диагоналях. Для восьми соседей центр равен 9, остальные коэффициенты равны −1. Значения округляются и ограничиваются диапазоном 0–255.

Метод подчеркивает мелкие детали и перепады яркости, но способен усилить шум и вызвать ореолы. Обработка каналов выполняется независимо; альфа-канал сохраняется.

4.2. Обработка границ

Clamp повторяет ближайший граничный пиксель. Reflect зеркально отражает изображение с повторением крайнего пикселя. Краевые пиксели не пропускаются и не заполняются черным цветом.

Для ядра 3 × 3 указанные реализации Clamp и Reflect могут совпадать: нужен только один слой соседей за границей, а Reflect здесь повторяет крайний пиксель.

4.3. Подготовка к пороговой обработке

Цветное изображение переводится в яркость: Y = round(0,299R + 0,587G + 0,114B). При Y > T пиксель становится белым, иначе — черным. Для глобальной обработки T одинаков для всего изображения, для адаптивной — зависит от положения пикселя.

4.4. Глобальный порог Оцу

Строится гистограмма яркости из 256 ячеек. Для каждого допустимого порога пиксели делятся на классы Y ≤ T и Y > T. Вычисляются численности классов и их средние яркости. Выбирается порог с максимальным разделением классов:

критерий(T) = $n_0 \cdot n_1 \cdot (\mu_0 - \mu_1)^2$.

Этот критерий пропорционален межклассовой дисперсии: постоянный для данного изображения нормирующий множитель не влияет на выбор максимума. Пустые классы пропускаются; при равном максимуме выбирается первый порог. Для однотонного изображения используется его яркость.

4.5. Итеративный глобальный порог

1. Начальный порог устанавливается равным средней яркости изображения.

2. Пиксели делятся на два класса: Y ≤ T и Y > T.

3. Находятся средние яркости классов μ_0 и μ_1.

4. Новый порог вычисляется как $T = (\mu_0 + \mu_1) / 2$.

5. Повторение прекращается при изменении порога менее 0,01, появлении пустого класса или достижении 100 итераций.

Подсчеты выполняются по гистограмме. Метод Оцу и итеративный метод используют разные правила выбора порога, поэтому на сложных распределениях яркости их результаты могут различаться.

4.6. Адаптивный порог

В подготовленной реализации порог определяется отдельно для каждого пикселя: **T(x, y) = среднее по окну − C**. Пользователь выбирает размер окна и поправку C. У границ окно пересекается с изображением, а сумма делится на фактическое число вошедших в него пикселей.

Для быстрого вычисления средних строится интегральное изображение. Каждая его ячейка хранит сумму яркостей прямоугольника от начала изображения. Сумма произвольного окна определяется четырьмя обращениями к таблице. После ее построения расчет порога требует постоянного числа операций на пиксель.

Ограничение: в методических указаниях требуется адаптивный алгоритм именно из лекции. Лекция не предоставлена. Соответствие выбранного локального среднего этому требованию необходимо проверить; при несовпадении алгоритм следует заменить.

5. Ход работы

1. Создан проект React + TypeScript с использованием Vite.

2. Определен пиксельный буфер: ширина, высота и массив RGBA. Доступ к пикселю выполняется по индексу (y · ширина + x) · 4 + номер канала.

3. Реализованы ручная обработка пикселей и стратегии Clamp / Reflect.

4. Реализованы два варианта повышения резкости с настройкой силы.

5. Добавлены преобразование в яркость, гистограмма и два метода глобального порога.

6. Реализован адаптивный порог через интегральное изображение.

7. Созданы элементы интерфейса: загрузка, выбор метода, настройки, области «До» и «После», сохранение PNG.

8. Добавлен фоновый обработчик и подготовлены синтетические тестовые изображения.

6. Порядок запуска и проверки

В папке проекта выполнить npm install, затем npm run dev. Открыть адрес из терминала. Для проверки типов и сборки выполнить npm run build. Команда npm run preview позволяет просмотреть production-сборку.

При проверке следует загрузить тестовое изображение, выбрать метод и параметры, нажать кнопку обработки и сохранить результат. Каждый метод применяется к исходному изображению, поэтому сравнение не зависит от ранее выполненных операций.

Тестовая база: размытые фигуры — для сравнения резкости; маленький яркий объект и изображение с тремя уровнями яркости — для сравнения глобальных порогов; темные символы на градиентном фоне — для сравнения глобального и адаптивного порога.

При сравнении глобальных методов необходимо указать цель сегментации: например, выделить только самый яркий объект или оба объекта. Подбор изображений и оценку качества нужно подтвердить фактическими результатами, а не только различием числовых порогов.

7. Результаты и ограничения проверки

Подготовлены исходный код, интерфейс и тестовая база. В среде подготовки материалов npm-сборка и испытания в браузере не выполнялись. Поэтому численные измерения времени, скриншоты и утверждения об экспериментально подтвержденном качестве в отчете отсутствуют.

Ниже приведен лист проверки для заполнения после запуска. Он не является протоколом уже проведенных испытаний.

Проверка	Что зафиксировать
Резкость: 4 и 8 соседей	Сила, скриншоты, детали и ореолы
Clamp и Reflect	Поведение на границах
Оцу и итеративный метод	Пороги и качество выделения объекта
Адаптивный порог	Размер окна, C, читаемость символов
Сохранение PNG	Совпадение файла с результатом на экране

Чтобы завершить экспериментальную часть, необходимо выполнить проверки, добавить снимки «До/После» и записать наблюдения. Для сдачи веб-приложение требуется разместить в общем доступе. Продвинутые режимы других вариантов и многопоточная обработка для максимальной оценки не реализованы.

8. Вывод

Подготовлена реализация веб-приложения для повышения резкости и пороговой обработки изображений. Реализованы два варианта Лапласиана, методы Оцу и итеративного порога, а также адаптивный порог через интегральное изображение. Разделены интерфейс, пиксельный доступ и вычислительные функции.

Различие подходов состоит в том, что фильтрация изменяет значения пикселей с учетом соседей, а пороговая обработка делит изображение на классы. Глобальные методы используют единый порог, адаптивный — локальный. Окончательные выводы о качестве конкретных результатов следует сделать после запуска и сравнения изображений; адаптивный алгоритм необходимо сверить с лекцией.